MedPhysTrack

Reference Guide: Phases 1–4 — What Was Built and Why

PHASE 1 — Foundation & Database Schema

Goal

Establish the project's codebase, connect it to a cloud database, and define the full data structure (every table the application will ever use) before writing any user-facing features.

What Was Created

- **GitHub repository** — the central, version-controlled home for all project code. Every change Claude Code makes can be tracked, reviewed, and rolled back here.
- **Project scaffold** — the skeleton of a React + Vite application: configuration files (`vite.config.js`), the HTML entry point (`index.html`), and core source files (`main.jsx`, `App.jsx`, `index.css`).
- **Supabase client connection** (`src/lib/supabase.js`) — the code that allows the website to talk to its database, reading connection details from environment variables rather than being hard-coded.
- `.env.example` **and** `.gitignore` — a template for secret configuration values, and a list of files/folders that should never be uploaded to GitHub (most importantly, the real `.env` file containing actual passwords and keys).
- **Database migration files** — two SQL scripts that build the database:

File	Purpose
`001_initial_schema.sql`	18 tables, indexes, and automatic updated_at timestamp triggers, plus a trigger that auto-creates a profile record when a new user signs up.
`002_rls_policies.sql`	Row Level Security (RLS) rules enabled on every table, with separate access rules for the super_admin, program_admin, and resident roles.

Steps You Performed

- Created a free account at supabase.com and a new project named 'medphystrack' (choosing a database password and server region).
- Located the project's Project URL and API keys under Project Settings → API Keys (Supabase had recently introduced new 'publishable'/'secret' key naming — the older 'anon public' and

'service_role' keys, found under the 'Legacy' tab, were used instead since that's what the generated code expected).

- Created a real `.env` file (copied from `.env.example`) and filled in the Supabase URL, anon/public key, and service_role key.
- Confirmed `.env` was listed in `.gitignore` so secrets are never pushed to GitHub.
- Ran `001_initial_schema.sql` in the Supabase SQL Editor — first attempt returned 'relation public.profiles does not exist' because the script had an internal ordering issue. Claude Code corrected the script, and the second attempt succeeded.
- Ran `002_rls_policies.sql` successfully afterward.
- Verified in the Supabase Table Editor that 18 tables were created, including a few extras beyond the original plan: application_files, inquiry_logs, minutes_files, minutes_members, template_modules, and template_tasks — the last two implementing the 'copy-on-provision' curriculum template design.

> *Why this matters: Row Level Security (RLS) is the database's own gatekeeper. Even though many residency programs will eventually share the same database, RLS ensures a user from one program can never see another program's data — this is what makes the application 'multi-tenant' safely.*

Glossary — Phase 1 Terms

Repository (repo)
A project folder tracked by Git/GitHub, storing the full history of every code change.

React + Vite
React is the toolkit used to build the visual interface; Vite is the fast build tool that compiles and serves that code during development.

Supabase
A hosted backend service providing a PostgreSQL database, user authentication, file storage, and security rules — without needing to run your own server.

Migration file
A SQL script that creates or changes database structure (tables, columns, rules) in a repeatable, trackable way.

Row Level Security (RLS)
Database-enforced rules that determine which rows of a table a given user is allowed to see or modify, based on their role and organization.

Environment variables (.env)
A file holding configuration values (URLs, secret keys) kept out of the codebase and out of GitHub for security.

Copy-on-provision

The design pattern where a master template curriculum is copied into a new program's own tables when that program is created, so each program can customize freely without affecting the original template.

PHASE 2 — Authentication & User Roles

Goal

Allow people to securely log in, and ensure each person is routed to the correct part of the application based on their role: Super Admin, Program Admin, or Resident.

User Creation Model Decided

- **Super Admin** (you/the developer) — created manually in Supabase, no signup form needed.
- **Program Admin** — created by the Super Admin when a new residency program signs up.
- **Resident** — created/invited by their Program Admin from within the app.
- Self-serve signup was intentionally left out of this phase; all accounts begin as admin-created and the new user sets their own password via an emailed link.

What Was Created

File	Purpose
`src/context/AuthContext.jsx`	Manages the logged-in session and user profile across the whole app; provides a useAuth tool other components use to check who is logged in.
`src/components/ProtectedRoute.jsx`	A wrapper that blocks a page from loading unless the visitor is logged in AND has the correct role for that page.
`src/pages/LoginPage.jsx`	Email + password sign-in form.
`src/pages/ForgotPasswordPage.jsx`	Sends a password-reset email via Supabase.
`src/pages/ResetPasswordPage.jsx`	Lets the user set a new password after clicking the emailed link.
`src/pages/dashboards/*.jsx`	Three placeholder dashboards, one each for Super Admin, Program Admin, and Resident, used to confirm routing works.
`src/App.jsx`	Ties all the routes/pages together and decides which dashboard to show.

How Routing Works

- Visiting the site's root address (`/`) checks whether the visitor is logged in and what role their profile has, then automatically sends them to `/super-admin`, `/program-admin`, or `/resident`.
- Each of those three sections is wrapped in ProtectedRoute, which checks the required role — a Resident trying to visit /super-admin is redirected back to their own dashboard rather than seeing

admin content.

- Any unrecognized address falls back to the root (`/`), which re-routes appropriately.

Steps You Performed

- In Supabase, went to Authentication → URL Configuration and added `http://localhost:5173/reset-password` to the allowed Redirect URLs list, so password-reset emails would work during local testing.
- Created the first Super Admin account manually: Authentication → Users → Add user, then edited that user's row in the profiles table to set `role = super_admin`.
- Ran the application locally with `npm run dev` (a command that starts a temporary local web server at `http://localhost:5173`) and confirmed:
  - Logging in as each of the three roles showed the correct dashboard.
  - Visiting another role's URL while logged in redirected back to the correct dashboard.
  - The 'forgot password' flow sent an email, and clicking the link allowed setting a new password — which then automatically logged the user in.
  - Logging out returned to the login page, and an incorrect password showed an error rather than crashing.

> *Why this matters: This phase established the security boundary between the three types of users. Every feature built afterward (Phases 3+) sits behind this login system, so a resident can never accidentally end up on an admin screen, and vice versa.*

Glossary — Phase 2 Terms

Authentication
The process of verifying who a user is (logging in), as opposed to (what they (what they're allowed to do once logged in).

Session
A temporary record that the browser is logged in as a particular user, so they don't have to re-enter their password on every page.

Role-based routing
Sending a logged-in user to a different part of the application depending on their assigned role (`super_admin` / `program_admin` / `resident`).

Protected route
A page that checks login status (and role) before displaying its content, redirecting elsewhere if the check fails.

localhost:5173
The address of the temporary copy of the website running on your own computer during development — only visible to you, not the public.

Profiles table

A database table (created in Phase 1) that stores each user's role and other profile information, linked to their Supabase login account.

PHASE 3 — Frontend Scaffold & Live Deployment

Goal

Build the application's visual shell — the layout, navigation, and page structure that all future features will live inside — and deploy the site to a real public URL so it can be tested and shared without running anything on your local computer.

What Was Created

File / Service	Purpose
`src/components/layout/AppLayout.jsx`	The shared page wrapper used by every screen in the app — renders the sidebar navigation, top bar, and a content area where each page's unique content appears.
`src/components/layout/Sidebar.jsx`	Left-side navigation menu. Renders different links depending on the logged-in user's role, so Super Admin, Program Admin, and Resident each see only their own section's navigation.
`src/pages/superadmin/SuperAdminDashboard.jsx`	Placeholder Super Admin home screen, later replaced with real content in Phase 4.
`Vercel (hosting service)`	A deployment platform that watches the GitHub repository and automatically rebuilds and republishes the site whenever new code is pushed. Free tier is sufficient for this project.
`medphystrack.com (custom domain)`	The public address of the live site, registered via Squarespace. DNS records were pointed at Vercel so that visiting medphystrack.com serves the application.

Steps You Performed

- Created a free account at vercel.com and connected it to the GitHub repository. Vercel automatically detected the React + Vite project type and configured the build settings without manual input.
- Added the Supabase environment variables (VITE_SUPABASE_URL and VITE_SUPABASE_ANON_KEY) to Vercel's project settings so the live site could connect to the database. These are the same values from the local `.env` file — Vercel stores them securely rather than putting them in the code.
- In Squarespace (where medphystrack.com is registered), added two DNS records pointing the domain at Vercel's servers: an A record for the root domain and a CNAME record for the www subdomain. Vercel automatically issued an SSL certificate (the padlock in the browser).

- Confirmed the live site was reachable at medphystrack.com and that logging in worked correctly — the same as local testing but on the public internet.
- Noted an important operational concern: new domains like medphystrack.com have no reputation history with corporate web filtering services (Zscaler, Cisco Umbrella, etc.). Residency programs hosted inside hospital networks may find the site blocked until their IT department adds it to an allowlist. An IT whitelist request template was identified as a standard customer onboarding artifact to prepare.

Why this matters: Deploying early — before most features exist — means every phase from here forward can be tested on the real live site, not just on a local machine. It also surfaces infrastructure problems (DNS, firewalls, SSL) early when they're cheap to fix.

Glossary — Phase 3 Terms

Vercel

A hosting platform that deploys web applications automatically from a GitHub repository. Every push to the main branch triggers a new build and deployment within about a minute.

DNS (Domain Name System)

The internet's address book — translates a human-readable domain name like medphystrack.com into the numeric IP address of the server that hosts the site.

SSL certificate

The credential that enables the padlock icon and https:// in the browser — it encrypts traffic between the user's browser and the server so passwords and data can't be intercepted in transit.

A record / CNAME record

Two types of DNS entries. An A record maps a domain name to a specific IP address. A CNAME record maps a domain name (e.g. www.medphystrack.com) to another domain name (e.g. Vercel's servers), letting Vercel manage the actual IP.

Environment variable (in Vercel)

The same concept as the local .env file — secret configuration values stored securely in Vercel's settings rather than in the code, injected into the build at deploy time.

Corporate web filter / allowlist

Enterprise networks (hospitals, health systems) often block access to unrecognized domains by default using services like Zscaler or Cisco Umbrella. An 'allowlist' request asks IT to explicitly permit medphystrack.com through the filter.

PHASE 4 — Program Management Admin UI

Goal

Give the Super Admin a real working interface to create and manage organizations and residency programs — replacing placeholder dashboards with functional screens. Establish the architectural patterns (data access layer, modal components, soft-delete) that all future phases will follow.

Key Architectural Decisions Made

- **Data access layer pattern:** All database calls live in dedicated API files under `src/lib/api/` rather than being written directly inside page components. This keeps pages readable and makes database logic easy to find and reuse.
- **Soft-delete only — no hard deletes, ever:** Instead of permanently removing organizations or programs, they are 'archived' (hidden from active views but fully preserved in the database). This is a non-negotiable constraint driven by CAMPEP accreditation requirements and ABR board eligibility — residents need their training records to remain intact for years after completing a program.
- **Audit logging in the database, not the application:** Every rename and archive action is recorded automatically by database triggers, not by the frontend code. This ensures changes made directly in the Supabase SQL editor are also captured — important for CAMPEP site visits where 'who changed what and when' may be asked.
- **Atomic program provisioning via RPC:** Creating a new program copies the full 13-module / 149-task curriculum template in a single database transaction (the provision_program stored procedure). If anything fails midway, the whole operation rolls back — no program can end up with a partial or missing curriculum.

What Was Created

Database changes:

Migration	Purpose
`status, archived_at, archived_by columns`	Added to both organizations and programs tables. status is 'active' or 'archived' (default active). archived_at and archived_by are stamped automatically by a trigger when status changes — the application never needs to set these manually.
`audit_log table`	Append-only log of every rename and archive/unarchive action. Columns: actor_id, action ('rename'/'archive'/'unarchive'), table_name, record_id, old_value (jsonb), new_value (jsonb), created_at. RLS allows Super Admin to read; no role can write directly — only triggers can insert.

`stamp_archive_metadata() trigger`	Before-update trigger on both tables. Automatically sets archived_at = now() and archived_by = current user when status changes to 'archived', and clears them when status returns to 'active'.
`log_organization_changes() trigger`	After-update trigger on organizations. Writes a row to audit_log whenever name or status changes.
`log_program_changes() trigger`	After-update trigger on programs. Same behavior as the organization trigger.

Frontend files:

File	Purpose
`src/lib/api/organizations.js`	listOrganizations(), getOrganization(), createOrganization(), updateOrganization(), archiveOrganization(), unarchiveOrganization(). listOrganizations() filters to active records by default; pass { includeArchived: true } to see archived ones.
`src/lib/api/programs.js`	listPrograms(), getProgram(), provisionProgram(), updateProgram(), archiveProgram(), unarchiveProgram(), getProgramCurriculum(). Program creation goes through the provision_program RPC rather than a direct insert.
`src/pages/superadmin/OrganizationsPage.jsx`	Table listing all active organizations with their linked program. Actions: Create (modal), Edit/rename (modal), Archive (confirmation modal).
`src/pages/superadmin/ProgramsPage.jsx`	Table listing all active programs with their parent organization. Actions: Create (modal), Edit/rename (modal), Archive (confirmation modal).
`src/components/modals/CreateOrganizationModal.jsx`	Form to create a new organization. Single name field.
`src/components/modals/CreateProgramModal.jsx`	Form to create a new program. Selects from organizations that don't yet have a program (enforced by the unique constraint on programs.org_id). Calls provision_program RPC.
`src/components/modals/EditOrganizationModal.jsx`	Pre-populated rename form. No-op (closes silently) if the name hasn't changed.
`src/components/modals/EditProgramModal.jsx`	Same pattern as EditOrganizationModal, for programs.

`src/components/modals/ArchiveOrganizationModal.jsx`	Requires typing the organization's exact name before the Archive button activates — prevents accidental archiving by misclick.
`src/components/modals/ArchiveProgramModal.jsx`	Same pattern as ArchiveOrganizationModal, for programs.

RLS Policy Cleanup

The Program Admin's original RLS policy on the programs table granted ALL permissions, which included DELETE — inconsistent with the soft-delete architecture. This was replaced with three explicit policies: SELECT, INSERT, and UPDATE only. The Super Admin policy retains ALL (Super Admins manage archive/unarchive). No application code changed — this was a database-only fix.

Steps You Performed

- Set up Claude Code in VS Code on Windows — resolved PATH and PowerShell execution policy issues that prevented it from running initially.
- Installed the Supabase CLI and captured a full schema snapshot (`supabase/schema_snapshot.sql`) as a reference for future phases.
- Created organizations and programs in the Super Admin UI and confirmed the full lifecycle: create → edit (rename) → archive, with audit log entries verified in the Supabase table editor.
- Learned to use Claude Code via the terminal ({code('claude')} command) to write and push multiple files at once — pasting a structured prompt rather than copying files manually.

> *Why this matters: The soft-delete + audit log infrastructure established here applies to every table in the system going forward. Resident task records, evaluation forms, and completion logs will all follow the same 'never delete, always preserve' principle — because a resident's proof of training for ABR board eligibility may be needed years after they leave a program.*

Glossary — Phase 4 Terms

Soft delete / Archive

Setting a status flag (status = 'archived') instead of removing a row from the database. The data is preserved permanently but hidden from normal application views. Opposite of a hard delete, which removes the row permanently.

Audit log

An append-only table that records who changed what and when. 'Append-only' means rows can be inserted but never updated or deleted — ensuring the history cannot be altered after the fact.

Database trigger

A function that the database runs automatically in response to an event (INSERT, UPDATE, DELETE) on a table — without the application having to call it explicitly. Used here to stamp archive metadata and write audit log entries.

RPC (Remote Procedure Call)

A named function stored in the database (a Postgres stored procedure) that the application can call by name. provision_program is an RPC that creates a program and copies its curriculum in a single atomic transaction.

Atomic transaction

A database operation where either everything succeeds or nothing is saved. If provision_program fails halfway through copying 149 tasks, the entire operation is rolled back and no incomplete data is left behind.

Data access layer

The set of files (`src/lib/api/`) that contain all database query logic. Pages call functions from this layer rather than writing SQL/Supabase queries directly, keeping the codebase organized and consistent.

Modal

A dialog box that appears over the current page (with a darkened background behind it) to collect input or confirm an action, without navigating away from the current screen.

CAMPEP

The Commission on Accreditation of Medical Physics Education Programs — the accrediting body for medical physics residency programs in the US. CAMPEP site visits may request documentation of curriculum, resident progress, and administrative changes, which is why audit logging and soft-delete are non-negotiable architectural constraints.

ABR (American Board of Radiology)

The certifying board for medical physicists. Residents must document their training history to establish board eligibility — sometimes years after completing their residency. This is why resident records must never be hard-deleted.